

Purchasing Intention Prediction Classifiers

Assignment Details

Module Title: Data and Network Mining

Module Code: B8IT156

Module Lecturer: Kunwar Madan

Students: Alex Vega - 20076089, Rory Patrick Sheridan - 20074605

Programme: Higher Diploma in Science in Data Analytics (Part Time)

Data:

Refer to dataset and its description at –

<https://archive.ics.uci.edu/ml/datasets/Online+Shoppers+Purchasing+Intention+Dataset>

Task:

This data belongs to an e-commerce company that wants to create a classifier to predict online shoppers' purchasing intention (i.e., whether an online shoppers' session will generate revenue or not).

Create an appropriate classifier by trying the following two classification algorithms in Python:

- a) Random Forest
- b) Adaptive Boosting

Github Notebook:

https://github.com/Ri-Dearg/data-mine-ca1/blob/main/data_mine_ca1.ipynb

Github Repository:

<https://github.com/Ri-Dearg/data-mine-ca1>

Google Colab:

<https://colab.research.google.com/drive/1SbAtbdi8gnPXGEQIXuYZq4gAtj8e3FZW>

Note:

The github notebook and Google Colab figures have minor differences in results due to computing conditions.

Abstract

This report develops and evaluates machine learning classifiers to predict the purchasing intention of online shoppers, where the target feature is *Revenue*. The dataset contains 12330 sessions with features such as duration, page visits and traffic source. Following structured data preparation, two ensemble methods were examined: Random Forest and Adaptive Boosting. In the development of both methods, we explored different techniques, methods, hyperparameters, and variables to identify the optimal performing model.

Recall was selected as the primary evaluation metric to minimise missed purchasing sessions, with an additional requirement of a precision threshold of 0.5 to ensure acceptable predictive reliability. Experimental results indicate that a Random Forest model achieved superior performance when utilising only the *PageValues* feature.

Notably, the *PageValues* feature dominates predictive performance in all models, with models using only this feature performing better or equally to models including other features. However, *PageValues* is a target-adjacent feature that is derived from historical conversion outcomes. As such, it acts as a proxy for the target feature. Therefore, it may inflate predictive performance, possibly limiting use in real-world deployment. This limitation will be explored further in the report.

Data Preparation

Data loading:

The dataset was loaded from the '*online_shoppers_intention.csv*' file into a pandas DataFrame. It involves importing the data into a structured format that can be easily manipulated, inspected, and processed by Python libraries. Without this step, no further analysis or model training is possible as the data would not be accessible in the programming environment.

Missing values:

Missing data can cause errors, distort training, or give biased results in models, so it is important to identify and handle them. The dataset was checked for nulls and no missing values were detected. No imputation of the data was necessary.

Duplicates:

Duplicate rows can lead to an overestimation of a model's performance and inflate performance when the same row is repeated across splits. Removing them ensures that each observation is unique and provides distinct information. 125 duplicate rows were identified and removed. Each row should represent a unique session. The rows have many specific duration values and categories which help with unique identification of the session. We concluded to identify the duplicates as repeated logging of sessions and remove them for modelling purposes, as having duplicate rows in both test and train could artificially inflate performance. We also confirmed the positive ratio of the target feature before and after removal, with a difference of less than 0.002, to ensure it would not skew the data.

Invalid values:

Invalid values can indicate data entry errors or corruption and may require special handling. We ran explicit validity checks, ensuring rate features were neither below 0 or above 1 and duration features were not below 0. No invalid values were detected under these checks.

Class Imbalance:

Understanding if the data is imbalanced is critical, as highly imbalanced datasets can lead to models that perform poorly on the minority class. The target variable, *Revenue*, was highly imbalanced. This knowledge informed our modelling design, using stratified splitting, SMOTE, exploring class weighting, and cross-validation later in the pipeline. Use of these techniques improved minority-class detection.

Categorical feature encoding:

Nominal categorical features were converted into numerical representations using one-hot encoding, as direct text-based categories like 'Month' (e.g., 'Jan', 'Feb') or 'VisitorType' (e.g., 'Returning_Visitor', 'New_Visitor') cannot be directly used. One-hot encoding creates new boolean (True or False in this version of pandas) columns for each unique category, converting categorical data into a format that models can utilise without implying ordinality between categories.¹ We also attempted to use one-hot encoding on the other discrete categorical features, such as Region, but found that it diminished performance, possibly due to increased dimensionality.²

Boolean to integer conversion:

The target variable was converted to integer type (0 or 1) for interpretability. Converting it to integers (1 for True, 0 for False) provides a numerical representation that tree methods easily process.

Feature scaling (StandardScaler):

We utilised SMOTE in many pipelines, and scaling is particularly necessary when utilising SMOTE as it uses nearest-neighbour distances. StandardScaler transforms the data such that each feature has a mean of 0 and a standard deviation of 1. This helps to ensure that all features contribute equally to the model's performance, preventing features with larger numerical ranges from disproportionately influencing the model. Tree models themselves do not require scaling. We implemented it within the pipeline so scaling occurs within cross-validation.

¹ (2019). One Hot Encoding in Machine Learning. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding/>.

²Wikipedia Contributors (2019). Curse of dimensionality. [online] Wikipedia. Available at: https://en.wikipedia.org/wiki/Curse_of_dimensionality.

Hyperparameter Tuning

Random Forest Hyperparameter Tuning

The Random forest classifier was tuned using GridSearchCV (5-fold CV). The selected pipeline included StandardScaler, SMOTE to optimise recall. Scaling was introduced to support SMOTE. As an alternative to SMOTE, we also evaluated class weighting to handle imbalance.

Hyperparameter Ranges

Search ranges included *n_estimators* [10–200], *max_depth* [None, 10, 20], *min_samples_leaf* [7–11], *min_samples_split* [2–4], and SMOTE *k_neighbors* [7–12]. These were chosen based on refinements after examining the best parameter results after evaluations. On Google Colab we have chosen more restrictive parameters due to computational limitations.

N_estimators

Refers to the number of decision trees in the forest. Increasing the number of estimators generally helps in reducing the variance of the model without increasing the bias. As more trees are added, the predictions become more stable and less sensitive to the noise in the training data, which reduces variance that individual decision trees might suffer from. An excessively small estimator value can lead to an underfit model.

Max_depth

Controls the maximum depth of each individual decision tree. This was tuned as deep trees can lead to overfitting, while very shallow trees might not capture enough complexity which leads to an underfitted model.

Min_samples_split

Controls the minimum number of samples required to split an internal node. Increasing this value reduces tree complexity and overfitting, improving generalisation.

Min_samples_leaf

Enforces a minimum number of samples per leaf; preventing overly specific patterns. This helps prevent “micro-leaves” that fit noise, and improves generalisation overall.

SMOTE - smote__k_neighbors

The choice of *k_neighbors* can significantly impact the quality of the synthetic samples generated. A small value might lead to generating samples that are too close to existing ones, potentially increasing overfitting. A large value can create more diverse samples but might also smooth out the decision boundary too much or introduce noise if the minority class is very sparse.

Adaptive Boosting Hyperparameter Tuning

The Adaptive Boosting classifier was tuned using GridSearchCV (5-fold CV). Experiments were run both with and without SMOTE and StandardScaler to support SMOTE within the pipeline. SMOTE was found to not contribute to model performance and so SMOTE hyperparameters were not included in the tuning grid.

Hyperparameter Ranges

Search ranges included Base estimator DecisionTreeClassifier with *max_depth* [1, 2], *n_estimators* [20-200], *learning_rate* [0.005-0.3]. These were chosen based on refinements after examining the best parameter results after evaluations. On Google Colab we have chosen more restrictive parameters due to computational limitations.

Classification__estimator

AdaBoost combines learners to arrive at its result. The estimator parameter controls the complexity of each learner. A decision stump, *max_depth*=1, enforces high bias and low variance, preventing overfitting. *max_depth*=2 can improve fit but may risk overfitting due to increased noise.

N_estimators

Controls the number of boosting rounds. More estimators can increase minority-class detection but also increase overfitting and false positives if the model becomes sensitive to noise.

Learning_rate

Controls the contribution of each learning tree to the final evaluation. Lower learning rates can improve stability and generalisation but may require more estimators. Higher learning rates may improve recall at the cost of increased false positives, lowering precision.

Overfitting Prevention

Prevention Mechanisms used for both Models

Train / Test split with stratification

Use of `train_test_split` with `stratify=y` to ensure the minority class rate is preserved in train vs test to enable fair evaluation and generalisation during testing. This reduces risk of misleading performance values from imbalanced data.

Cross-validation during tuning

Both models are tuned using `GridSearchCV` with 5-fold CV to reduce the risk of overfitting on a single split. This helps provide a more stable estimate. This checks that the chosen hyper-parameters can generalise across different data samples.

SMOTE and Scaling inside Pipeline

SMOTE and (StandardScaler when applicable) are used inside of the pipeline within cross-validation to ensure there is no data-leakage across the folds. Scaling parameters are generated using the training data, rather than the full dataset, ensuring consistency.

Varied Feature Set Tests

Models were tested with various feature sets to analyse robustness and reduce risk of a single feature-dependent setup. Reduced performance under other feature sets can identify models that do not generalise well under different conditions.

Random Forest Overfitting Prevention Mechanisms

Random Forest mitigates overfitting by averaging predictions across many trees to reduce variance. In our implementation, to aid generalisation, we used `max_features='sqrt'`.³ This ensures that each tree considers only a random subset of features for splitting which introduces diversity and prevents trees from only focusing on the same set of features. As noted in [Random Forest Hyperparameter Tuning](#), we used various controls (e.g. `max_depth`, `min_samples_split`, `min_samples_leaf`) to prevent overly deep or specific splits that could train on noise.

Adaptive Boost Overfitting Prevention Mechanisms

AdaBoost can overfit when weak learners become too complex or boosting is too aggressive, becoming overly sensitive to noise. We mainly mitigated this by using various controls as listed in [Adaptive Boosting Hyperparameter Tuning](#) (e.g. `base_estimator` `n_estimators`, `learning_rate`). Imbalance techniques, such as SMOTE were evaluated, however they provided no performance benefit over the predictive strength of the `PageValues` feature (recall 0.798, precision 0.560). Therefore overfitting prevention focused mainly on these parameters rather than resampling.

³ StackExchange Contributors (2020). grid search result `max_features = 'sqrt'` in random forest - how to understand. [online] Data Science Stack Exchange. Available at: <https://datascience.stackexchange.com/questions/82560/grid-search-result-max-features-sqrt-in-random-forest-how-to-understand> [Accessed 6 Mar. 2026].

Model Evaluation

Chosen Metric

We decided that recall was the most suitable metric as the business objective is to identify as many revenue-generating sessions as possible. This metric measures the ability of the model to avoid false negatives. We also chose to have a precision constraint of at least 0.5 to avoid unusable models. It was important to report precision and F1-score to contextualise the recall-precision trade-off. F1 was not chosen because it prioritises balancing precision and recall, but business objectives target recall. Due to the class imbalance in Revenue, accuracy can be misleading because high accuracy can be achieved by predicting the majority class.

In this context, a 'false negative' occurs when the model predicts that a user session will not generate revenue, but the user actually does make a purchase. Each false negative represents a missed opportunity for the business to engage with an interested customer, potentially through targeted marketing, special offers, or improved user experience. Maximizing recall addresses this by ensuring that as many actual revenue generating sessions as possible are identified. The cost associated with a false negative (a lost potential sale) is often higher than the cost of a false positive (identifying a non-revenue session as revenue generating). If outreach cost is low and missing a buyer is costly, recall is prioritised.

By identifying a high proportion of actual revenue generating sessions, the business can then apply resources (e.g., personalized recommendations, retargeting campaigns, etc) to these sessions. Even if some of these efforts are directed at false positives, the overall strategy is to not let genuine buyers slip through the cracks, and then optimize the engagement strategy to convert the identified high-potential sessions.

Model Comparison

Model Selection

With a recall of 0.809 the Random Forest model with SMOTE + StandardScaler, trained only on *PageValues* was chosen as the model for deployment. While the Random Forest model using class weighting achieved a higher recall of 0.825, its precision is lower at 0.539 compared to 0.576. While the metric difference appears minor, this translates to 42 more false positives for 6 additional true positives, which may incur a much higher operational cost. The most performant Adaptive Boosting model has slightly lower class metrics across the board.

However, performance was highly influenced by *PageValues* to the extent it was the only feature that led to usable models. The issue with *PageValues* is that it is a target-adjacent feature derived from historical conversion behaviour, and so deployment of these models requires consideration of this fact. In a real world context, it would be better to not deploy any of these models, and utilise other methodologies to achieve results without the use of *PageValues*.⁴ This will be discussed in the next section, [PageValues Influence](#). Additionally, further comparison and tests can be viewed in the [Appendix](#).

Random Forest:

Random Forest consistently performed well across varied feature sets, excluding runs without *PageValues*. The model trained only on *PageValues* delivered recall of 0.809, precision at 0.576 and f1 of 0.673. When evaluating broader feature sets, the recall achieved for the positive class (Revenue=True) was lower (≈ 0.793 on the full feature set and ≈ 0.791 – 0.793 on the refined set, depending on the execution environment). Notably, the model employing class weighting without SMOTE or a scaler achieved the highest recall of 0.825 for predicting revenue, with a precision of 0.539 and an F1-score of 0.652 with less training-time overhead than the best scaled SMOTE model. If training resources are limited, it would be a practical resource-conscious alternative.

Adaptive Boosting:

This pipeline consistently achieved a stable test set recall of 0.798, precision of 0.560, f1 of 0.658, for the positive class in tested feature sets that include *PageValues*. Whether SMOTE, StandardScaler, both, or neither were used, the results remained the same in these sets, suggesting that these preprocessing steps did not contribute to performance as the predictive influence of *PageValues* dominates the sets.. Feature sets without *PageValues* were capable of achieving the highest recall score of 0.908, but the models with higher recall inversely impacted precision to an unacceptable degree (below 0.5 constraint) with lows of 0.209. Overall, AdaBoost was competitive but did not exceed the selected Random Forest model on the chosen deployment criteria.

⁴ Liu, J. and Liu, C. (2026). Prediction of Online Shoppers' Purchasing Intention Based on SHAP-IGWO-EM Method. International Journal of Swarm Intelligence Research, [online] 17(1). doi:<https://doi.org/10.4018/IJSIR.397667>.

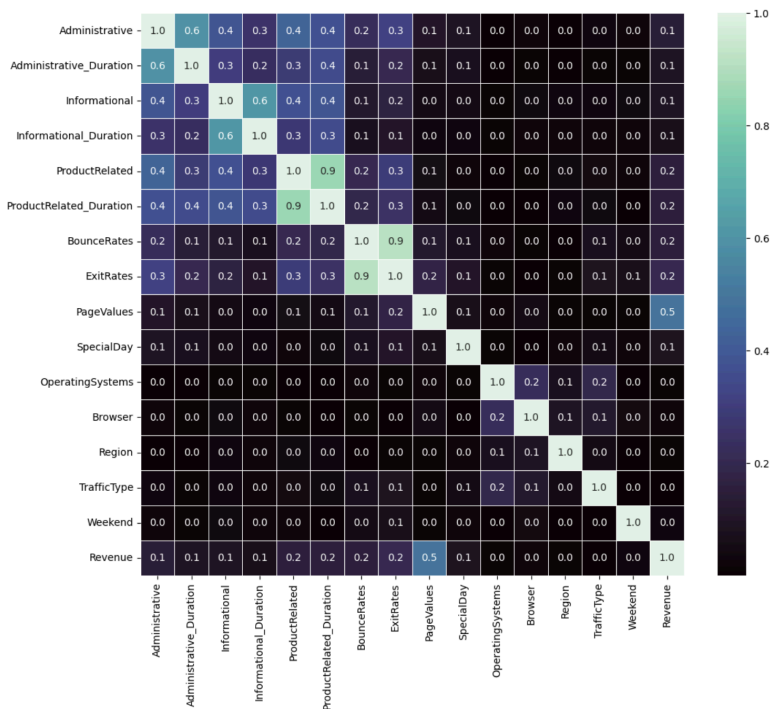
PageValues Influence

Correlation

The *PageValues* feature showed to be a particularly important feature across both models tested. There was a strong correlation between *PageValues* and Revenue (Pearson $r = \sim 0.5$), and both Random Forest and Adaptive Boosting models identified this value as the most influential feature in predicting Revenue.

In UA Google Analytics, page value is the estimated revenue generated by a page. The metric is calculated with the formula (Transaction Revenue + Total Goal Value) / Unique Pageviews.⁵ UA has been deprecated by Google⁶, and this metric has been discontinued in GA4. The feature that is predicting Revenue is itself derived from the target value of Revenue, so it already indicates how likely a purchase is to occur.. This observation is supported by the experiments, where models trained using only *PageValues* performed as well as or better than models trained on the full feature set. We have selected a model for deployment based on our results but in the truest real-world status, models based on *PageValues* are impractical for current analytics.

Feature Correlation



Importances

Random Forest	Adaptive Boosting
PageValues0.561470 Administrative0.063218 ProductRelated_Duration0.061526 ExitRates0.052735 Month_Nov0.047764 Administrative_Duration0.034820 ProductRelated0.026254 TrafficType0.023140 OperatingSystems0.018525 BounceRates0.015348 Region0.013968 Informational0.012440 Browser0.012044 Month_May0.011680 VisitorType_Returning_Visitor0.010629 Month_Mar0.007624 SpecialDay0.007430 Informational_Duration0.007422 Weekend0.004356	PageValues0.178533 Administrative0.000000 Weekend0.000000 VisitorType_Other0.000000 Month_Sep0.000000 Month_Oct0.000000 Month_Nov0.000000 Month_May0.000000 Month_Mar0.000000 Month_June0.000000 Month_Jul0.000000 Month_Feb0.000000 Month_Dec0.000000 TrafficType0.000000 Administrative_Duration0.000000 Region0.000000 Browser0.000000 OperatingSystems0.000000 SpecialDay0.000000

⁵ Google (2012). Understanding And Using Page Value. [online] Google Analytics Solutions. Available at: <https://analytics.googleblog.com/2012/07/understanding-and-using-page-value.html> [Accessed 6 Mar. 2026].

⁶ Google (2023). Upgrade to Google Analytics 4 before July 1. [online] Google. Available at: <https://blog.google/products/ads-commerce/upgrade-to-google-analytics-4-before-july-1/>.

We decided to run experiments across both models to determine the impact of this feature in this dataset. In the table, we can see that it dominates the importance for both Random Tree and Adaptive Boosting.

Results

When '*PageValues*' was excluded from the Random Forest model, the recall significantly dropped from approximately 0.79 to 0.565. Conversely, training a Random Forest model with only '*PageValues*' as a feature resulted in a recall of 0.809. This recall was slightly higher than the model trained on the full feature set (0.793), suggesting that '*PageValues*' alone accounts for a substantial portion of the predictive power.

In the case of Adaptive Boosting, the metrics of the model with the full feature set (recall of 0.798, precision of 0.560) were identical to the model utilising only *PageValues*, indicating that the other features did not contribute any predictive power to those models. Removing '*PageValues*' from AdaBoost with SMOTE led to a higher recall score of 0.908. This came at a severe cost to precision, which dropped to ~0.20-0.22. Without '*PageValues*' and without SMOTE, the AdaBoost model's recall dropped to 0.013, demonstrating near zero detection of the target Feature.

Conclusion

As page value is no longer available for use in Google Analytics, it is important to recognise that without this metric, the utility of both models decreases drastically with recall, precision, or both dropping to an unusable level. The models either fail to identify a much larger proportion of genuinely interested customers, or identify too many false positives, which will lead to lost sales and revenue. The dataset provided relies heavily on this single metric that captures behaviour that is strongly associated with purchase outcomes, and so carries a significant business risk. In the future for this business, it may be important to create and adapt robust internal metrics to offset this over-reliance.

Appendix

Adaptive Boost Test Table

Experiment	Pipeline	CV Recall	Precision	Recall	F1	Accuracy	TN	FP	FN	TP
Full set	Only AdaBoost	0.8027	0.560	0.798	0.658	0.870	1819	240	77	305
Full set	Only SMOTE	0.8080	0.560	0.798	0.658	0.870	1819	240	77	305
Full set	Only Scaler	0.8027	0.560	0.798	0.658	0.870	1819	240	77	305
Full set	Scaler + SMOTE	0.8080	0.560	0.798	0.658	0.870	1819	240	77	305
Only PageValues	Scaler + SMOTE	0.8080	0.560	0.798	0.658	0.870	1819	240	77	305
Only PageValues	Only AdaBoost	0.8027	0.560	0.798	0.658	0.870	1819	240	77	305
No PageValues	Only AdaBoost	0.0124	0.500	0.013	0.026	0.844	2054	5	377	5
No PageValues	Only SMOTE	0.8872	0.221	0.882	0.353	0.494	870	1189	45	337
No PageValues	Scaler + SMOTE	0.9062	0.219	0.898	0.352	0.483	836	1223	39	343
No PageValues	Scaler + SMOTE	0.846* *CV=F1	0.321	0.531	0.400	0.751	1630	429	179	203
Refined Set No PageValues	Scaler + SMOTE	0.9062	0.209	0.908	0.340	0.447	744	1315	35	347

Random Forest Test Table

Experiment	Pipeline	CV Recall	Precision	Recall	F1	Accuracy	TN	FP	FN	TP
Full set	Scaler + SMOTE	0.7916	0.577	0.793	0.668	0.877	1837	222	79	303
Full set	Only RF	0.5760	0.726	0.584	0.647	0.900	1975	159	84	223
Refined set	Scaler + SMOTE	0.7922	0.572	0.791	0.664	0.875	1833	226	80	302
No PageValues	Scaler + SMOTE	0.5183	0.327	0.565	0.415	0.750	1615	444	166	216
Only PageValues	Scaler + SMOTE	0.7968	0.558	0.785	0.652	0.869	1821	238	82	300
Class weighting (no SMOTE)	class weight	0.8394	0.539	0.825	0.652	0.862	1790	269	67	315

Note on SMOTENC

SMOTENC is typically a more appropriate technique to use in the pipeline than SMOTE when mixing both numerical and categorical features.⁷ However, a follow-up experiment was conducted using SMOTENC to examine whether this change would have materially impacted our results. It was found that the differences were negligible with only incremental differences, or identical results. The notebook with these results can be found here:

https://github.com/Ri-Dearg/data-mine-ca1/blob/main/data_smotenc.ipynb

Use of AI Tools

AI Tools were used as a supplementary resource during the development of this assignment. The experimental design, code implementation, model evaluation, and final conclusions were independently developed by the students.

The use of AI included:

- Clarifying ML methods, techniques and concepts.
- Guidance and brainstorming for the modelling process and understanding results.
- Guidance for coherent structure in experimentation.
- Nuances of SMOTE vs SMOTENC and details of specific hyperparameters.
- Generating tables of statistics, and two pieces of code that are highlighted by comments

⁷ Imbalanced-learn.org. (2024). SMOTENC — Version 0.13.0. [online] Available at: https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTENC.html?utm_source=chatgpt.com.